VIT BHOPAL
Vityarthi Project Report

- **School:** SCAI
- **Course:** CSE1021 – Introduction to Problem solving and Programming
- **Programming language:** Python
- **Student Name:** Harshit Manshani
- **Register Number:** 26BAI10900
- **Academic Year:** 2026

Introduction

The Risk Analyzer & Recommendation system is a python-based project designed to analyze academic and study-related information provided by students. The system evaluates factors such as marks, attendance, study habits, assignment completion, and exam preparation.

Based on the entered information, the system calculates a risk score and classifies the student as Low Risk, Medium Risk, or High Risk. It also identifies the factors contributing to the risk and provides suitable recommendations to help the student improve their academic performances.

Problem Statement

Students may have problems such as low marks, poor attendance, backlogs, or irregular study habits, and inadequate exam preparation. However, it may be difficult for students to identify which factors are affecting their academic performance.

The Risk Analyzer & Recommendation System addresses this problem by analyzing student academic and study-related information. It calculates a risk score, identifies the student's risk level, highlights the factor causing the risk, and provides recommendations for improvement.

Objectives

- To collect academic and study-related information from student.
- To Validate the information entered by the student.
- Analyze the collected information.
- Calculate a risk score based on predefined conditions.
- Classify the student into Low Risk, Medium Risk, or High Risk.
- Identify the individual factors contributing to the risk.
- Generate recommendations based on the identified risk factors.
- Display the final risk report to the user.

Non-functional Requirements

- **Usability** – the system should be simple and easy for students to use.

- **Reliability** – the system should provide consistent results for the same input values.

- **Accuracy** – the system should correctly calculate the risk score according to the predefined rules.

- **Maintainability** – the program should be divided into separate modules so that it can be easily modified and maintained.

- **Performance** – the system should process the entered information and display the result quickly.

System Architecture

The system follows a modular architecture where different Python files perform different tasks.

Module Description

- **Input Handler** – Collects student information.
- **Validation** – Checks whether the entered information is valid.
- **Risk Analyzer** – Calculates the risk score and risk level.
- **Recommendation** – Generates recommendations based on risk factors.
- **Report** – Displays the final result.
- **Utils** – Contains supporting functions.

Architecture Flow

Student

|

↓

Input Handler Module

|

↓

Validation Module

|

↓

Risk Analyzer Module

|

↓

Recommendation Module

|

↓

Report Module

|

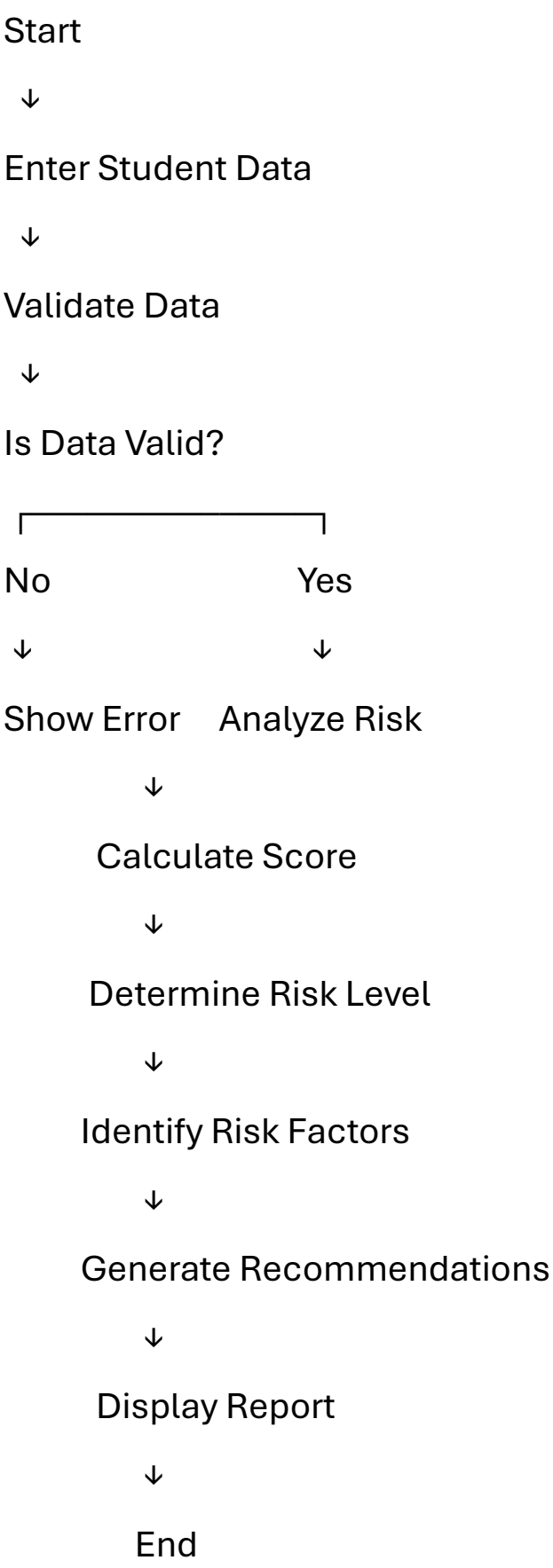
↓

Final Risk Report

Process / Workflow

- The project follows these steps:
- The student enters academic and study-related information.
- The system validates the entered information.
- The validated information is sent to the risk analyzer.
- The system checks predefined risk conditions.
- Risk points are added for identified risk factors.
- The total risk score is calculated.
- The system determines the risk level.
- The system identifies the factors contributing to the risk.
- Recommendations are generated.
- The final student risk report is displayed.

Workflow

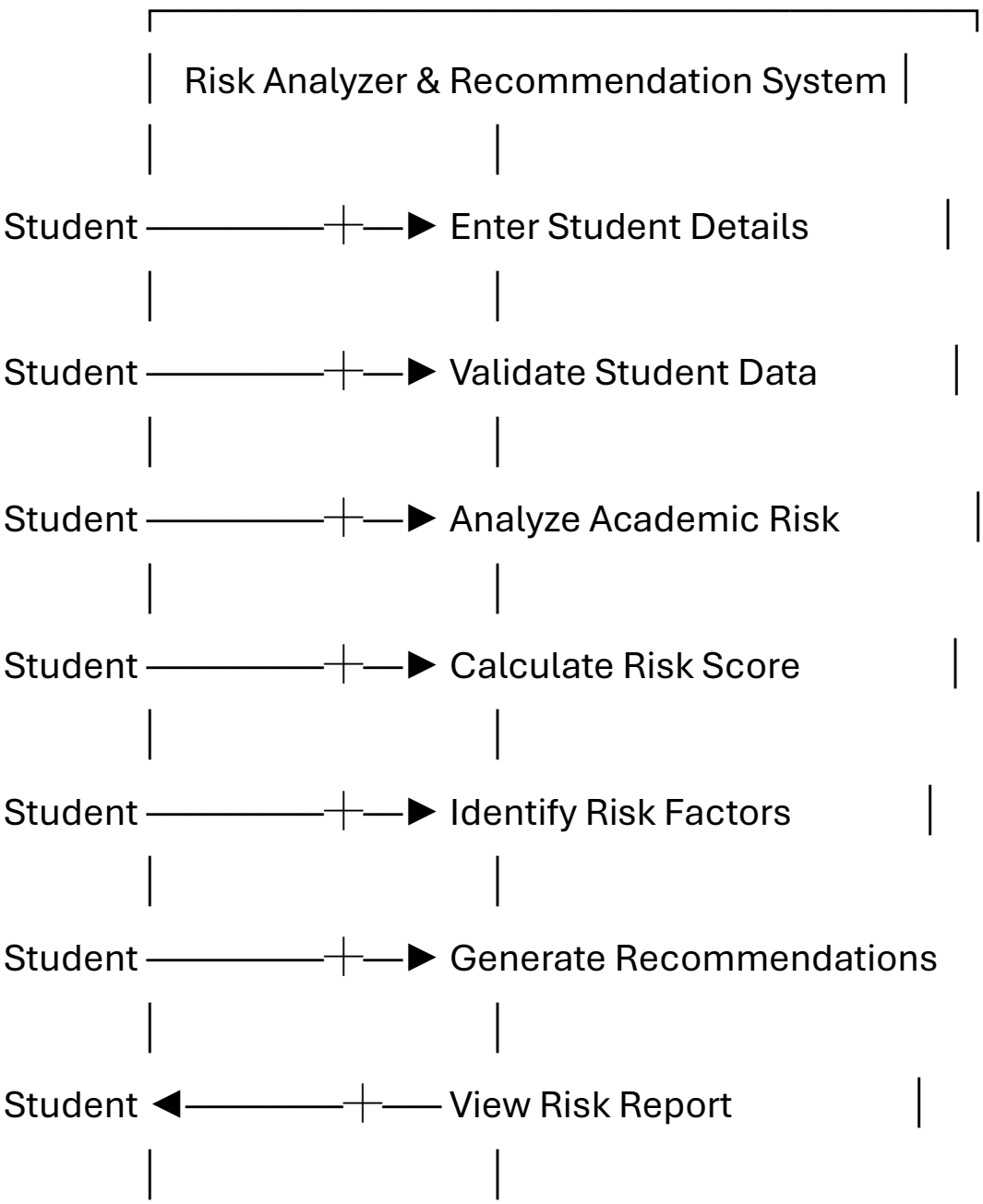

Start
Enter Student Data
Validate Data
Is Data Valid?
No
Yes
Show Error
Analyze Risk
Calculate Score
Determine Risk Level
Identify Risk Factors
Generate Recommendations
Display Report
End

Use Case Diagram

Main Actor- Student

Student can:

- Enter academic information
- Enter study-related information
- View risk score
- View risk level
- View risk factors
- View recommendations
- View final report


Risk Analyzer & Recommendation System
Student
Enter Student Details
Student
Validate Student Data
Student
Analyze Academic Risk
Student
Calculate Risk Score
Student
Identify Risk Factors
Student
Generate Recommendations
Student
View Risk Report

Class / Component Diagram

Since this project is implemented using separate Python modules, the major components are:

Components

- main.py
- input_handler.py
- validation.py
- risk_analyzer.py
- recommendation.py
- report.py
- utils.py

Input Handler

↓

Validation

↓

Risk Analyzer

↓

Recommendation

↓

Report

Utils → Supports the main program

Main → Controls the overall workflow

Sequence Diagram

The sequence of operations is:

The student first provides the required information. The system validates the information and then analyzes the student's risk. After calculating the risk level, recommendations are generated and the final report is displayed.

Student → Input Handler → Validation

Student ← Input Handler

Validation → Main

Main → Risk Analyzer

Risk Analyzer → Main

Main → Recommendation

Recommendation → Main

Main → Report

Report → Student

Design Decisions and Rationale

The project uses a modular design so that different tasks are handled by separate Python files.

A rule-based scoring system was selected because it is simple, understandable, and suitable for the project's purpose. Each identified risk factor contributes a fixed number of points to the total risk score.

Each risk factor is assigned **10 points**. Based on the total score, the system classifies the student as:

- **0–40:** Low Risk
- **41–80:** Medium Risk
- **81 or above:** High Risk

This approach makes the risk calculation easy to understand and allows the rules to be modified easily in the future.

Implementation Details

- The project is implemented using Python and is divided into multiple modules.

- **main.py**
  Controls the overall workflow of the program and connects the different modules.

- **input_handler.py**
  Collects academic and study-related information from the student.

- **validation.py**
  Checks whether the entered information is within valid ranges and whether the selected options are valid.

- **risk_analyzer.py**
  Analyzes the student information, calculates the risk score, identifies risk factors, and determines the risk level.

- **recommendation.py**
  Generates recommendations based on the identified risk factors.

- **report.py**
  Displays the risk score, risk level, risk factors, and recommendations.

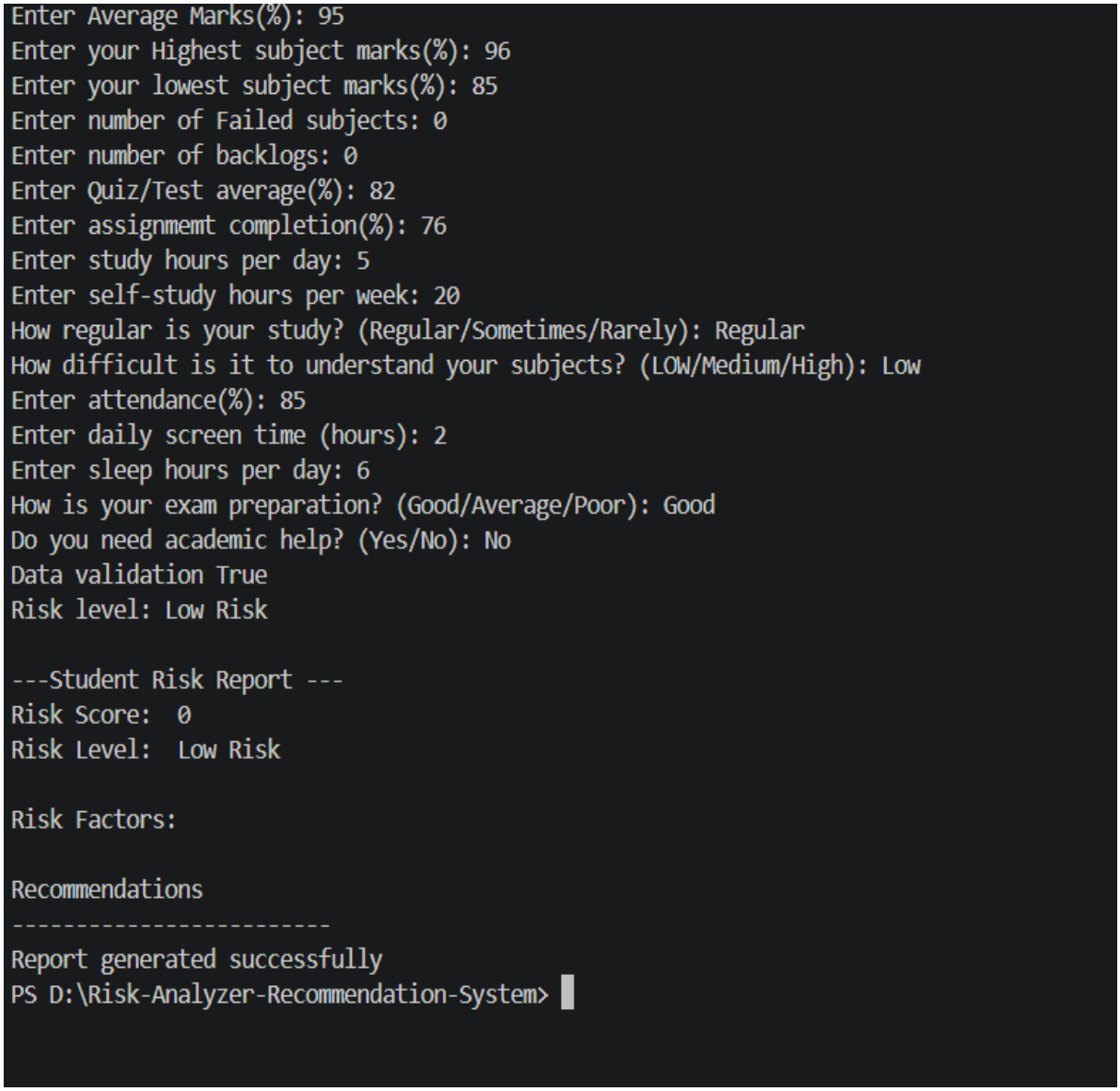
- **utils.py**
  Contains helper functions used by the project.

Screenshots and Results

The project was tested using different types of student data.

Low Risk Result

- The system was tested with values representing a student with relatively good academic and study conditions.



Medium Risk Result

- The system was tested with values that triggered several risk factors.

```
Enter your lowest subject marks(%): 40
Enter number of Failed subjects: 1
Enter number of backlogs: 1
Enter Quiz/Test average(%): 55
Enter assignmemt completion(%): 60
Enter study hours per day: 1
Enter self-study hours per week: 5
How regular is your study? (Regular/Sometimes/Rarely): Sometimes
How difficult is it to understand your subjects? (LOW/Medium/High): Medium
Enter attendance(%): 70
Enter daily screen time (hours): 6
Enter sleep hours per day: 6
How is your exam preparation? (Good/Average/Poor): Average
Do you need academic help? (Yes/No): No
Data validation True
Risk level: Medium Risk

---Student Risk Report ---
Risk Score:  70
Risk Level:  Medium Risk

Risk Factors:
- Low Average Marks
- Failed subjects
- Backlogs
- Low Attendance
- Low assignment completion
- Low study hours
- Low self-study hours

Recommendations
- Focus on Failed subjects and seek extra academic support
- Clear your backlogs by giving extra attention to pending subjects
- Complete assignments on time and regularly.
- Increase your daily study time gradually.
- increase your weekly self-study time.
------------------------
Report generated successfully
PS D:\Risk-Analyzer-Recommendation-System>
```

High Risk Result

- The system was tested with values representing a student with relatively bad academic and study conditions.

```
Enter Average Marks(%): 40
Enter your Highest subject marks(%): 45
Enter your lowest subject marks(%): 30
Enter number of Failed subjects: 2
Enter number of backlogs: 2
Enter Quiz/Test average(%): 40
Enter assignmemt completion(%): 50
Enter study hours per day: 1
Enter self-study hours per week: 4
How regular is your study? (Regular/Sometimes/Rarely): Rarely
How difficult is it to understand your subjects? (LOW/Medium/High): High
Enter attendance(%): 60
Enter daily screen time (hours): 8
Enter sleep hours per day: 5
How is your exam preparation? (Good/Average/Poor): Poor
Do you need academic help? (Yes/No): Yes
Data validation True
Risk level: High Risk

---Student Risk Report ---
Risk Score:  130
Risk Level:  High Risk

Risk Factors:
- Low Average Marks
- Low highest subject marks
- Very low marks in a subject
- Failed subjects
- Backlogs
- Low Attendance
- Low quiz/test average
- Low assignment completion
- Low study hours
- Low self-study hours
- Irregular study
- Difficulty understanding subjects
- Poor exam preparation
```

```
Recommendations
 - Focus on Failed subjects and seek extra academic support
 - Clear your backlogs by giving extra attention to pending subjects
 - Complete assignments on time and regularly.
 - Increase your daily study time gradually.
 - increase your weekly self-study time.
 - Follow a regular study schedule.
 - Ask teachers or classmates for help with difficult topics.
 - Start preparing for exams early and revise regularly.
------------------------
Report generated successfully
PS D:\Risk-Analyzer-Recommendation-System>
```

Testing Approach

- The project was tested using different input cases to check whether the system works correctly.

Test Case 1 – Low Risk

Purpose: Check whether the system identifies a low-risk student.

Risk Score:

Risk Level: Low Risk

Result: Low Risk was generated successfully.

Test Case 2 – Medium Risk

Purpose: Check whether multiple risk factors are correctly added to the score.

Risk Score: 60

Risk Level: Medium Risk

Test Case 3– High Risk

Purpose: Check whether multiple risk factors are correctly added to the score.

Risk Score:

Risk Level: High Risk

Challenges

During the development of the project, some challenges were encountered:

- Organizing the program into multiple Python modules.
- Connecting the different modules correctly.
- Ensuring that user input was validated correctly.
- Handling different types of student inputs.
- Designing suitable rules for calculating the risk score.
- Testing different risk levels and invalid inputs.

Learnings

Through this project, the following concepts were learned:

- Python functions and modules.
- Conditional statements.
- Loops.
- Dictionaries and lists.
- User input handling.
- Data validation.
- Modular programming.
- Rule-based problem solving.
- Testing and debugging.
- Organizing a Python project into multiple files.
- Using GitHub for project submission.

Future Enhancements

The project can be improved in the future by:

- Adding more academic and behavioural factors.
- Providing more detailed recommendations.
- Adding a graphical user interface.
- Storing previous student reports.
- Adding data visualization.
- Using historical student data for more advanced analysis.
- Developing a machine-learning-based prediction system in the future.

References

- Python documentation
- VITyarthi project guidelines
- CSE1021 – Introduction to Problem Solving and Programming course material

---